SecureGate IAM / Zero Trust Gateway Engine

Documento de arquitetura e plano de implementação · versão 0.1 · outubro de 2026

1. Visão geral

O **SecureGate IAM** é uma plataforma corporativa de gestão de identidade e acesso baseada no princípio **Zero Trust**: nenhuma requisição é confiável por padrão, e toda decisão de acesso considera identidade, dispositivo, localização e nível de risco no momento da requisição.

A plataforma centraliza:

- **Provisionamento** de usuários, grupos e papéis.
- **Emissão de tokens** (OAuth 2.0 / OpenID Connect) com claims dinâmicas, calculadas por políticas de acesso contextual.
- **Auditoria e resposta a eventos de segurança em tempo real**, via mensageria assíncrona.
- **Gestão de segredos e credenciais temporárias** via HashiCorp Vault.

1.1 Objetivos

1. Fornecer um Authorization Server compatível com OAuth 2.0 e OIDC, com JWT assinado em RS256 e rotação de chaves.
2. Avaliar políticas de acesso condicional (RBAC + ABAC) a cada autenticação e a cada decisão de autorização.
3. Emitir eventos de segurança que alimentem, de forma assíncrona, bloqueios e revogações em um cache distribuído.
4. Manter trilha de auditoria imutável, particionada e consultável com bom desempenho.
5. Ser totalmente reproduzível localmente (Docker Compose) e validada por CI/CD.

1.2 Fora de escopo (versão 1)

- Federação com provedores externos (SAML, Google, Azure AD) – previsto como evolução.
- Interface administrativa completa (a v1 expõe apenas API REST; um painel simples é opcional).
- Autenticação biométrica / WebAuthn (evolução futura).

1.3 Requisitos não funcionais de referência

Requisito	Meta inicial
Latência de `/oauth2/token` (p95)	< 200 ms a 300 req/s
Latência de introspecção/validação via cache (p95)	< 20 ms
Propagação de revogação de token	< 2 s entre evento e bloqueio efetivo
Disponibilidade dos serviços stateless	Escala horizontal sem estado local
Cobertura de testes (domínio e aplicação)	⩾ 80%

Essas metas servem de critério de aceite dos testes de carga (seção 9).

2. Decisões de stack

O enunciado permite Java/Spring Boot ou NestJS. Recomendação: **Java 21 + Spring Boot 3.3+**, pelos motivos abaixo.

Decisão	Escolha	Justificativa
Linguagem/Framework	Java 21, Spring Boot 3	Ecossistema maduro de segurança, Spring Authorization Server, Spring Security, Spring Cloud Vault
Authorization Server	**Spring Authorization Server** (customizado)	Implementa OAuth 2.1/OIDC; permite customizar claims, consentimento e armazenamento
Banco relacional	PostgreSQL 16	Particionamento declarativo, JSONB, maturidade
Cache/sessões	Redis 7	Estruturas com TTL, pub/sub, baixa latência
Mensageria	RabbitMQ 3.13	Topologia flexível (exchanges, DLQ, retry), boa integração Spring AMQP
Segredos	HashiCorp Vault	Segredos dinâmicos, Transit, PKI

Decisão	Escolha	Justificativa
Migrações	Flyway	Versionamento do schema e das partições
Testes	JUnit 5, Mockito, Testcontainers, ArchUnit	Unitários, integração com infraestrutura real, regras de arquitetura
Carga	Apache JMeter	Requisito do projeto
Observabilidade	Micrometer + Prometheus + Grafana, OpenTelemetry	Métricas e tracing distribuído
CI/CD	GitHub Actions	Requisito do projeto

> **Nota:** se preferir NestJS, a arquitetura permanece a mesma; mudam as bibliotecas (por exemplo `oidc-provider` no lugar do Spring Authorization Server, Jest no lugar de JUnit).

3. Arquitetura

3.1 Visão de componentes

A solução é composta por serviços independentes que se comunicam por HTTP (síncrono) e RabbitMQ (assíncrono):

1. **iam-core** – Authorization Server, gestão de identidades, papéis, políticas e emissão de eventos. É o serviço principal.
2. **policy-enforcer** (consumidor de eventos) – consome filas de segurança e atualiza o Redis com bloqueios, revogações e sessões.
3. **gateway** (opcional na v1, recomendado na v2) – Spring Cloud Gateway validando JWT e consultando o Redis para revogação, implementando o “Zero Trust Gateway”.
4. **Infraestrutura** – PostgreSQL, Redis, RabbitMQ, Vault, Prometheus/Grafana.

```
flowchart LR
  Client[Cliente / Aplicação] -->|OAuth2 / OIDC| Core[iam-core]
  Client -->|Bearer JWT| GW[Gateway]
  GW -->|valida JWT + checa revogação| Redis[(Redis)]
  Core --> PG[(PostgreSQL)]
  Core --> Vault[(HashiCorp Vault)]
  Core -->|publica eventos| MQ{{RabbitMQ}}
  MQ --> Enforcer[policy-enforcer]
  Enforcer --> Redis
  Enforcer --> PG
  Core -->|sessões ativas| Redis
```

3.2 Contextos delimitados (DDD)

O `iam-core` é um **monólito modular** com fronteiras estritas, extraíveis para serviços no futuro:

Contexto	Responsabilidade	Agregados principais
Identity	Usuários, credenciais, MFA, ciclo de vida	`User`, `Credential`, `MfaFactor`
Access	Papéis, permissões, grupos, atributos	`Role`, `Permission`, `Group`
Policy	Regras de acesso condicional, avaliação de risco	`AccessPolicy`, `RiskAssessment`
Authorization (OAuth/OIDC)	Clientes, tokens, consentimentos, chaves	`RegisteredClient`, `Authorization`, `SigningKey`
Audit	Registro imutável de eventos de segurança	`AuditEvent`
Session	Sessões ativas e dispositivos	`Session`, `Device`

A comunicação entre contextos ocorre por **interfaces (portas)** e **eventos de domínio**, nunca por acesso direto a repositórios de outro contexto. A regra é verificada automaticamente com ArchUnit.

3.3 Clean Architecture – camadas por contexto

Cada contexto segue a mesma estrutura de pacotes:

```
com.securegate.iam.<contexto>
 ├─ domain          # entidades, value objects, eventos de domínio, serviços
de domínio (sem Spring)
 ├─ application     # casos de uso, portas de entrada/saída (interfaces),
DTOs internos
 ├─ infrastructure  # adaptadores: JPA, Redis, RabbitMQ, Vault, clientes HTTP
 └─ presentation    # controllers REST, handlers de exceção, mapeadores de
request/response
```

Regra de dependência: `presentation → application → domain` e `infrastructure → application/domain`. O `domain` não depende de nenhum framework.

3.4 Estrutura do repositório

```
securegate/
 ├─ iam-core/                # Spring Boot (monólito modular)
 ├─ policy-enforcer/         # Spring Boot (consumidor)
 ├─ gateway/                 # Spring Cloud Gateway (fase final)
 ├─ libs/
 |   └─ security-events/     # contrato compartilhado de eventos (schemas)
 ├─ infra/
 |   ├─ docker-compose.yml
 |   ├─ vault/               # políticas e script de init
 |   ├─ rabbitmq/            # definitions.json (exchanges, filas, bindings)
 |   ├─ postgres/
 |   └─ observability/       # prometheus.yml, dashboards Grafana
 ├─ load-tests/jmeter/       # planos .jmx e relatórios
 ├─ docs/                    # ADRs, diagramas, runbooks
 └─ .github/workflows/       # pipelines de CI/CD
```

4. Segurança e controle de acesso

4.1 OAuth 2.0 / OIDC

Fluxos suportados na v1:

- **Authorization Code + PKCE** – aplicações web e mobile (fluxo principal).
- **Client Credentials** – comunicação máquina a máquina.
- **Refresh Token** com rotação e detecção de reuso.

Fluxos deliberadamente não suportados: Implicit e Resource Owner Password (descontinuados no OAuth 2.1).

Endpoints: `/.well-known/openid-configuration`, `/oauth2/authorize`, `/oauth2/token`, `/oauth2/jwks`, `/oauth2/revoke`, `/oauth2/introspect`, `/userinfo`.

4.2 Tokens JWT e chaves

- Assinatura **RS256** com par de chaves RSA de 3072 bits (mínimo 2048).
- Cada chave possui `kid`; o endpoint JWKS publica as chaves vigentes e a anterior durante a janela de rotação.
- **Rotação** automática agendada (ex.: a cada 90 dias), com sobreposição para não invalidar tokens emitidos.
- A chave privada fica protegida: preferencialmente via **Vault Transit** (a assinatura ocorre no Vault e a chave nunca sai dele); alternativa na fase inicial é armazenar a chave cifrada, com a chave de cifra no Vault.
- Access tokens de vida curta (5–15 min); refresh tokens com rotação.
- Claims customizadas: `roles`, `tenant`, `risk_level`, `device_id`, `session_id` (`sid`), `amr` (métodos de autenticação) e `acr` (nível de garantia).

4.3 Acesso condicional (Conditional Access)

O contexto **Policy** avalia, a cada tentativa de login e a cada emissão de token, um **contexto de acesso**:

Sinal	Exemplos
Identidade	Usuário, grupos, papéis, estado da conta, MFA realizado
Dispositivo	Conhecido/registrado, fingerprint, conformidade declarada
Rede/localização	IP, país (GeoIP), faixa de rede corporativa, “viagem impossível”
Comportamento	Falhas recentes, horário atípico, volume de requisições
Recurso	Cliente OAuth, escopo solicitado, sensibilidade

Resultado da avaliação (`Decision`): `ALLOW`, `ALLOW_WITH_MFA` (step-up), `ALLOW_LIMITED` (escopos reduzidos, TTL menor) ou `DENY`, acompanhado da lista de políticas que determinaram o resultado (para auditoria e explicabilidade).

Modelo de política (exemplo em JSON, armazenado em JSONB):

```
{
  "name": "Admin exige MFA fora da rede corporativa",
  "priority": 10,
  "target": { "roles": ["ADMIN"] },
  "conditions": [
    { "attribute": "network.corporate", "operator": "EQ", "value": false }
  ],
  "effect": "ALLOW_WITH_MFA"
}
```

Algoritmo: políticas ordenadas por prioridade; `DENY` explícito sempre prevalece; ausência de política aplicável resulta em `DENY` (default deny). A pontuação de risco (0–100) é calculada por regras simples e determinísticas na v1 (soma ponderada de sinais), deixando o motor extensível para modelos mais sofisticados.

4.4 RBAC + ABAC

- **RBAC:** usuários pertencem a grupos e papéis; papéis agregam permissões no formato `recurso:ação` (ex.: `invoice:read`).
- **ABAC:** políticas podem referenciar atributos do sujeito, do recurso e do ambiente (ex.: `user.department == resource.department`).
- A avaliação ABAC usa uma expressão restrita e segura (por exemplo Spring Expression Language em modo somente leitura e *sandboxed*, ou um DSL próprio em JSON). **Nunca executar expressões arbitrárias vindas de entrada de usuário sem restrição de contexto.**

4.5 Integração com HashiCorp Vault

Uso	Mecanismo Vault
Segredos da aplicação (senha do banco, credenciais RabbitMQ)	KV v2 + Spring Cloud Vault
Credenciais temporárias de banco	Database Secrets Engine (usuário com TTL)
Assinatura de JWT / criptografia de campos sensíveis	Transit Secrets Engine
Autenticação do serviço no Vault	AppRole (ambiente local/CI) ou Kubernetes auth (produção)
Credenciais temporárias para recursos privilegiados (PAM)	Endpoint do `iam-core` que solicita lease ao Vault, com TTL e auditoria

O modo `dev` do Vault é aceitável apenas no Docker Compose local; o `README` deve documentar claramente que produção exige unseal, políticas mínimas e TLS.

4.6 Proteção de credenciais e hardening

- Senhas com **Argon2id** (ou bcrypt com custo ≥ 12) e *pepper* opcional armazenado no Vault.
- Bloqueio progressivo e *rate limiting* por IP e por conta (Redis, janela deslizante).
- MFA por TOTP (RFC 6238) na v1; códigos de recuperação com hash.
- Cabeçalhos de segurança, CORS restritivo, validação rigorosa de `redirect_uri` (correspondência exata).
- Comparações de segredos em tempo constante; mensagens de erro de login sem enumeração de usuários.
- Nenhum segredo, token ou senha em logs (filtro de mascaramento obrigatório).
- Análise de dependências (Dependabot/OWASP Dependency-Check) e SAST (CodeQL) na pipeline.

5. Mensageria e eventos de segurança

5.1 Topologia RabbitMQ

- **Exchange** `security.events` (tipo *topic*), durável.
- **Routing keys** no padrão `<categoria>.<evento>`, por exemplo `auth.login.suspicious`, `auth.login.failed`, `token.revoked`, `session.terminated`, `policy.changed`, `user.locked`.
- **Filas:**
  - `enforcer.block-policy` → bindings `auth.login.suspicious`, `user.locked`, `policy.changed`
  - `enforcer.revocation` → bindings `token.revoked`, `session.terminated`
  - `audit.persist` → binding `#` (todos os eventos, para a trilha de auditoria)
- **Resiliência:** cada fila possui *dead-letter exchange* (`security.events.dlx`) e fila de retry com TTL; confirmações do publicador (*publisher confirms*) e *acks* manuais no consumidor.

5.2 Contrato de evento

Todos os eventos usam um envelope versionado:

```
{
  "eventId": "uuid-v7",
  "type": "auth.login.suspicious",
  "version": 1,
  "occurredAt": "2026-10-17T14:03:11Z",
  "correlationId": "uuid",
  "actor": { "userId": "...", "clientId": "..." },
  "context": { "ip": "203.0.113.10", "deviceId": "...", "country": "BR" },
  "data": { "riskScore": 82, "reasons": ["impossible_travel", "new_device"] }
}
```

O esquema é mantido na biblioteca compartilhada `libs/security-events`, evitando divergência entre produtor e consumidores.

5.3 Garantias e padrões

- **Transactional Outbox:** o `iam-core` grava o evento numa tabela `outbox` na mesma transação da mudança de estado; um publicador assíncrono envia ao RabbitMQ. Isso evita perda de eventos entre commit e publicação.
- **Entrega *at-least-once*** → consumidores **idempotentes**, usando `eventId` (chave de deduplicação no Redis com TTL, e restrição única no banco para auditoria).
- **Ordenação:** não garantida globalmente; eventos de revogação carregam timestamp e o consumidor aplica "o mais recente vence".

5.4 Consumidor `policy-enforcer` e uso do Redis

Chave Redis	Conteúdo	TTL
`session:{sid}`	Dados da sessão ativa (usuário, dispositivo, expiração)	Igual à sessão
`user:sessions:{userId}`	Conjunto de `sid` ativos	Dinâmico
`revoked:jti:{jti}`	Marca de token revogado	Até a expiração natural do token
`revoked:sid:{sid}`	Sessão revogada (invalida todos os tokens da sessão)	Vida máxima do refresh token
`block:user:{userId}`	Bloqueio temporário ou permanente	Conforme a política
`block:ip:{ip}`	Bloqueio de IP	Conforme a política

Chave Redis	Conteúdo	TTL
`ratelimit:{scope}:{key}`	Contadores de rate limiting	Janela
`dedupe:event:{eventId}`	Controle de idempotência	24 h

O gateway e o `iam-core` consultam essas chaves para decidir em milissegundos se um token (mesmo ainda válido criptograficamente) deve ser rejeitado.

6. Persistência e desempenho (PostgreSQL)

6.1 Modelo de dados (visão resumida)

Tabela	Observações
`users`, `credentials`, `mfa_factors`	Identidade; credenciais em tabela separada
`roles`, `permissions`, `role_permissions`, `user_roles`, `groups`, `group_members`	RBAC
`access_policies`	Políticas em JSONB, versionadas, com `enabled` e `priority`
`oauth_clients`, `oauth_authorizations`, `oauth_consents`, `signing_keys`	OAuth/OIDC
`sessions`, `devices`	Sessões e dispositivos conhecidos
`outbox_events`	Transactional outbox
`audit_events`	**Particionada**, somente inserção

6.2 Particionamento da trilha de auditoria

- `audit_events` com **particionamento declarativo por faixa (RANGE) em `occurred_at`**, uma partição por mês.
- Chave primária composta `(id, occurred_at)`, exigência do Postgres para tabelas particionadas.
- Criação automática das partições futuras (job agendado na aplicação ou `pg_partman`) e política de retenção (detach/arquivamento de partições antigas).

- Índices por partição: `(actor_user_id, occurred_at DESC)`, `(event_type, occurred_at DESC)` e, para busca por payload, índice GIN em JSONB apenas se necessário.
- Imutabilidade: o usuário da aplicação recebe apenas `INSERT` e `SELECT` nessa tabela; opcionalmente, encadeamento de hash entre registros para evidência de adulteração.

6.3 Otimização com Hibernate/JPA

- Evitar o problema N+1: `@EntityGraph`, `JOIN FETCH` pontuais e projeções (DTO/interface) para leituras.
- Paginação por *keyset* (cursor) nas consultas de auditoria, em vez de `OFFSET` em tabelas grandes.
- *Batch inserts* (`hibernate.jdbc.batch_size`) e `StatelessSession`/JDBC puro para ingestão de auditoria em volume.
- `@Version` para controle otimista em agregados concorrentes (ex.: sessões, refresh tokens).
- Pool HikariCP dimensionado conforme carga, com métricas habilitadas.
- Uso de `EXPLAIN (ANALYZE, BUFFERS)` para validar os planos das consultas críticas, documentando resultados no repositório.

7. Infraestrutura e Docker

7.1 Serviços do Docker Compose

Serviço	Imagem/Origem	Portas (local)	Observação
`postgres`	postgres:16	5432	Volume nomeado, *healthcheck*
`redis`	redis:7	6379	Persistência AOF opcional
`rabbitmq`	rabbitmq:3.13-management	5672 / 15672	Carrega `definitions.json`
`vault`	hashicorp/vault	8200	Modo dev local + script de init
`iam-core`	build local	8080	Depende de postgres, redis, rabbitmq, vault
`policy-enforcer`	build local	8081	Depende de rabbitmq, redis
`prometheus` / `grafana`	imagens oficiais	9090 / 3000	Observabilidade

7.2 Boas práticas de imagem

- *Multi-stage build*; imagem final baseada em JRE slim (Temurin 21), executando como usuário não root.
- Configuração por variáveis de ambiente (12-factor); nenhum segredo embutido na imagem.
- `depends_on` com `condition: service_healthy` e *actuator health probes* (`/actuator/health/liveness` e `/readiness`).
- Perfis do Compose (`profiles`) para subir apenas infraestrutura (desenvolvimento) ou a stack completa.

8. Estratégia de testes

Nível	Ferramentas	Foco
Unitários	JUnit 5, Mockito, AssertJ	Domínio (políticas, risco, value objects) e casos de uso

Nível	Ferramentas	Foco
Arquitetura	ArchUnit	Regras de dependência entre camadas e contextos
Integração	Spring Boot Test + **Testcontainers** (Postgres, Redis, RabbitMQ, Vault)	Repositórios, mensageria, fluxo OAuth completo
Contrato	Testes de esquema dos eventos	Compatibilidade produtor/consumidor
Segurança	Testes negativos (token adulterado, `alg=none`, redirect inválido, reuso de refresh)	Garantir que ataques comuns falham
Carga	JMeter	Desempenho e concorrência (seção 9)

As migrações Flyway são executadas nos testes de integração, validando também o schema e o particionamento.

9. Testes de carga (JMeter)

9.1 Cenários

1. **Token endpoint** (`client_credentials`) – throughput máximo e latência.
2. **Login + Authorization Code + PKCE** – fluxo completo com usuários distintos (CSV de massa de dados).
3. **Validação/introspecção de token** com e sem consulta de revogação no Redis.
4. **Rajada de falhas de login** – verificação de rate limiting e bloqueio, e sua propagação via fila.
5. **Escrita de auditoria sob carga** – impacto da ingestão na latência dos endpoints.

9.2 Perfil de execução

- *Ramp-up* gradual (ex.: 0 → 300 usuários virtuais em 2 minutos), patamar de 10 minutos, *ramp-down*.
- Execução em modo não GUI (`jmeter -n -t plano.jmx -l resultado.jtl -e -o relatorio/`).
- Coleta de métricas da aplicação (Prometheus/Grafana) correlacionadas com o resultado do JMeter.

9.3 Critérios de aceite

Erro < 0,5%; p95 e throughput dentro das metas da seção 1.3; ausência de crescimento contínuo de memória; filas RabbitMQ sem acúmulo sustentado.

10. CI/CD (GitHub Actions)

10.1 Pipeline de integração contínua (`ci.yml`)

Disparada em *pull requests* e *push* para `main`:

1. **Checkout** e configuração do JDK 21 com cache Gradle/Maven.
2. **Build e testes unitários** (`./mvnw verify` ou `./gradlew check`).
3. **Testes de integração** com Testcontainers (o runner `ubuntu-latest` já fornece Docker).
4. **Qualidade e segurança:** CodeQL, verificação de dependências, análise de cobertura (JaCoCo com limite mínimo).
5. **Build das imagens Docker** (sem publicar em PRs) e *scan* de vulnerabilidades da imagem (ex.: Trivy).
6. **Artefatos:** relatórios de teste e cobertura.

10.2 Pipeline de entrega (`cd.yml`)

- Disparada em *tags* `v*.*.*` ou merge na `main`.
- Publica imagens no GitHub Container Registry (GHCR) com tag de versão e SHA.
- *Smoke test*: sobe o Docker Compose no runner e valida os endpoints de saúde e o JWKS.
- Execução opcional e manual (`workflow_dispatch`) do plano JMeter reduzido.

10.3 Governança

- Proteção da branch `main`: revisão obrigatória e *status checks* verdes.
- Convenção de commits (Conventional Commits) e versionamento semântico.
- Segredos do pipeline em *GitHub Secrets*; nunca no repositório.

11. Observabilidade e operação

- **Métricas:** taxa de logins por resultado, decisões de política (`ALLOW` / `DENY`), tamanho das filas, latência de consumo, *hit ratio* do Redis, uso do pool de conexões.

- **Logs estruturados (JSON)** com `correlationId`, `userId` (quando aplicável) e sem dados sensíveis.
- **Tracing:** OpenTelemetry propagando contexto também pelas mensagens do RabbitMQ.
- **Alertas sugeridos:** pico de `auth.login.suspicious`, DLQ não vazia, falha de rotação de chaves, indisponibilidade do Vault.

12. Plano de implementação por etapas

Cada etapa termina com **entregáveis verificáveis** e um critério de conclusão (*Definition of Done*). Estimativas assumem desenvolvimento individual em tempo parcial e servem apenas como referência.

Etapa 0 – Fundação do repositório *(~2 dias)*

- Criar o repositório, estrutura de pastas (seção 3.4), `README` inicial, `.gitignore`, `CODEOWNERS` e ADR-001 (decisão de stack).
- Configurar build multi-módulo (Maven ou Gradle) e formatação/lint (Spotless, Checkstyle).
- **Conclusão:** projeto compila, `./mvnw verify` roda com um teste de exemplo.

Etapa 1 – Infraestrutura local com Docker Compose *(~2 dias)*

- Compose com PostgreSQL, Redis, RabbitMQ e Vault, com *healthchecks*.
- `definitions.json` do RabbitMQ (exchange, filas, DLX) e script de init do Vault (KV, Transit, AppRole).
- **Conclusão:** `docker compose up` sobe a infraestrutura saudável; painéis acessíveis.

Etapa 2 – Esqueleto do `iam-core` e CI inicial *(~3 dias)*

- Aplicação Spring Boot com Actuator, estrutura de pacotes por contexto, tratamento global de erros (Problem Details, RFC 9457).
- Flyway com migração base; regras ArchUnit; conexão com Postgres via Testcontainers.
- Primeira versão do `ci.yml` (build + testes unitários + integração).
- **Conclusão:** pipeline verde no GitHub; teste de integração sobe contexto com banco real.

Etapa 3 – Contexto Identity e Access *(~1 semana)*

- Modelagem de `User`, credenciais (Argon2id), papéis, permissões e grupos.

- API REST de provisionamento (CRUD de usuários, atribuição de papéis), validações e auditoria básica.
- Autenticação da própria API administrativa (token bootstrap inicial).
- **Conclusão:** provisionamento completo testado (unitário + integração), cobertura ≥ 80% nesses contextos.

Etapa 4 – Authorization Server OAuth 2.0 / OIDC *(~1,5 semana)*

- Integração do Spring Authorization Server com persistência JPA customizada de clientes, autorizações e consentimentos.
- Fluxos Authorization Code + PKCE, Client Credentials e Refresh Token com rotação.
- JWT RS256, endpoint JWKS, `kid`, claims customizadas (`roles`, `sid`).
- Rotação de chaves e integração com Vault (Transit ou chave cifrada).
- **Conclusão:** um cliente de teste obtém tokens válidos; JWT verificável pelo JWKS; testes negativos de segurança passam.

Etapa 5 – Políticas de acesso condicional e risco *(~1,5 semana)*

- Contexto Policy: modelo de política em JSONB, motor de avaliação (default deny, prioridade, `DENY` prevalece).
- Coleta de sinais (IP, GeoIP, dispositivo, falhas recentes) e cálculo de risco.
- Integração do resultado no fluxo de login/emissão de token (step-up com TOTP, escopos reduzidos, negação).
- RBAC + ABAC nas decisões de autorização.
- **Conclusão:** matriz de cenários (tabela de decisão) coberta por testes parametrizados.

Etapa 6 – Eventos de segurança e Outbox *(~1 semana)*

- Biblioteca `security-events` com envelope e tipos de evento.
- Tabela `outbox_events`, publicador assíncrono, *publisher confirms*, retry e DLQ.
- Emissão dos eventos: login suspeito/falho, token revogado, sessão encerrada, mudança de política, usuário bloqueado.
- **Conclusão:** eventos aparecem nas filas corretas; falha do broker não perde eventos (teste de resiliência).

Etapa 7 – Microsserviço `policy-enforcer` e Redis *(~1 semana)*

- Consumidores idempotentes atualizando bloqueios, revogações e sessões no Redis.
- Consulta de revogação no `iam-core` (filtro) e *rate limiting* distribuído.
- Endpoint de logout/revogação ponta a ponta.

- **Conclusão:** revogar um token faz com que ele seja rejeitado em menos de 2 s; teste de integração end-to-end com Testcontainers.

Etapa 8 – Auditoria particionada e otimização *(~1 semana)*

- `audit_events` particionada por mês (migração Flyway + criação automática de partições).
- Consumidor `audit.persist` com *batch insert*; API de consulta com paginação por cursor.
- Revisão de N+1, índices e planos de execução; documentar ganhos medidos.
- **Conclusão:** consultas de auditoria com tempo estável sobre milhões de linhas (massa gerada por script).

Etapa 9 – Vault avançado e PAM *(~1 semana)*

- Credenciais dinâmicas de banco e segredos via Spring Cloud Vault.
- Endpoint de credenciais temporárias privilegiadas (lease com TTL, justificativa obrigatória, auditoria completa).
- **Conclusão:** aplicação opera sem segredos estáticos em arquivos; leases expiram e são renovados corretamente.

Etapa 10 – Testes de carga com JMeter *(~1 semana)*

- Planos `.jmx`, massa de dados, execução não GUI e relatórios HTML.
- Métricas correlacionadas no Grafana; ajuste de pools, TTLs e índices.
- **Conclusão:** metas da seção 1.3 atingidas ou desvios documentados com plano de correção.

Etapa 11 – Dockerização completa, CD e hardening final *(~1 semana)*

- Dockerfiles multi-stage, perfis do Compose, `cd.yml` com GHCR e *smoke tests*.
- CodeQL, Trivy, revisão contra OWASP ASVS (checklist nível 2 como referência) e *threat model* resumido (STRIDE).
- **Conclusão:** `docker compose --profile full up` sobe tudo; release `v1.0.0` publicada.

Etapa 12 – Gateway Zero Trust e documentação final *(opcional, ~1 semana)*

- Spring Cloud Gateway validando JWT, checando revogação no Redis e aplicando políticas por rota.

- ADRs finalizados, diagramas, *runbooks*, coleção Postman/Bruno e vídeo/GIF de demonstração.
- **Conclusão:** demonstração de ponta a ponta: login → acesso → evento suspeito → bloqueio → acesso negado.

Resumo do cronograma

Fase	Etapas	Resultado
Fundação	0–2	Infraestrutura, esqueleto e CI funcionando
Núcleo de identidade	3–5	Provisionamento, OAuth/OIDC e acesso condicional
Tempo real	6–7	Eventos, consumidor e revogação via Redis
Dados e segredos	8–9	Auditoria escalável e Vault avançado
Qualidade e entrega	10–12	Carga, hardening, CD e gateway

Total estimado: **cerca de 12 a 14 semanas** em tempo parcial.

13. Riscos e mitigações

Risco	Impacto	Mitigação
Complexidade do Spring Authorization Server ao customizar persistência e claims	Atraso na Etapa 4	Começar com a persistência padrão em JDBC e migrar para JPA depois; testes de fluxo completo desde o início
Perda ou duplicação de eventos	Revogações não aplicadas	Outbox + idempotência + DLQ + testes de resiliência
Consistência eventual (token válido por alguns instantes após revogação)	Janela de exposição	TTL curto de access token + checagem de revogação no Redis; documentar a janela aceitável
Gestão incorreta de chaves/segredos	Comprometimento de tokens	Vault Transit, rotação automatizada, nenhum segredo em repositório ou imagem

Risco	Impacto	Mitigação
Escopo excessivo	Projeto não concluído	Etapa 12 e federação externa são opcionais; entregar o núcleo primeiro
Particionamento mal planejado	Degradação de consultas	Medir com massa de dados realista na Etapa 8 antes de consolidar o modelo

14. Definição de pronto do projeto (v1.0)

- [ ] `docker compose up` sobe toda a stack saudável com um único comando.
- [ ] Fluxos Authorization Code + PKCE, Client Credentials e Refresh Token funcionando com JWT RS256 e JWKS.
- [ ] Acesso condicional decidindo `ALLOW`, `ALLOW_WITH_MFA`, `ALLOW_LIMITED` e `DENY` com motivos auditáveis.
- [ ] Eventos de segurança publicados via Outbox e consumidos de forma idempotente; revogação efetiva em < 2 s.
- [ ] Auditoria em tabela particionada, imutável e consultável por cursor.
- [ ] Segredos e credenciais temporárias gerenciados pelo Vault.
- [ ] Pipeline de CI com testes unitários, integração, análise de segurança e cobertura mínima.
- [ ] Relatório JMeter anexado com metas atingidas.
- [ ] Documentação (README, ADRs, diagramas, runbook) completa.

15. Próximo passo

A sequência recomendada é iniciar pela **Etapa 0 e Etapa 1** (repositório, estrutura multi-módulo e `docker-compose` com Postgres, Redis, RabbitMQ e Vault), pois todas as etapas seguintes dependem dessa base. Antes de começar, três decisões em aberto:

1. **Build tool:** Maven ou Gradle (Kotlin DSL)?
2. **Linguagem/stack:** confirmar Java 21 + Spring Boot 3 (recomendado) ou NestJS.
3. **Assinatura JWT:** iniciar com Vault Transit desde a Etapa 4 ou começar com chave local cifrada e migrar depois?